



Tarea Corta 2

Tecnológico de Costa Rica
Escuela de Ingeniería en Computación
Redes
Segundo Semestre 2026

1. Objetivo General

→ Implementar una calculadora de subredes IPv4 utilizando el lenguaje de programación GNU C y desplegarla mediante Kubernetes y Helm.

2. Objetivos Específicos

- Desarrollar habilidades en el uso de sockets, operadores bitwise y el lenguaje de programación GNU C.
 - Implementar un protocolo de comunicación para el intercambio de información entre un cliente y un servidor.
-

3. Datos Generales

- El valor de la tarea es: 10%
- La tarea debe ser implementada en grupos de máximo 5 personas.
- Cualquier indicio de copia será calificado con una nota de 0 y será procesado de acuerdo con el reglamento. La copia incluye código/configuraciones que se puede encontrar en Internet y que sea utilizado parcial o totalmente sin el debido reconocimiento al autor.
- La revisión se realiza de forma presencial mediante una evaluación escrita, para cuya resolución deben estar presentes todos los integrantes del grupo.
- Se debe incluir documentación con instrucciones claras para ejecutar el proyecto.
- Se espera que todos y todas las integrantes del grupo entiendan la implementación suministrada.
- Se deben seguir buenas prácticas de programación en el caso de que apliquen. Por ejemplo, documentación interna y externa, estándares de código, diagramas de arquitectura, diagramas de flujo, pruebas unitarias son algunas de las buenas prácticas que se esperan de un estudiante de Ingeniería en Computación.
- El lenguaje de implementación, en caso de ser requerido y no especificado por el profesor, deberá ser seleccionado por cada grupo y debe quedar debidamente documentado. No se permite el uso de Python.

- Toda documentación debe ser implementada en Markdown.
 - Si la instalación de servicios no se encuentra automatizado se obtendrá una nota de 0. Si el proyecto no se entrega completo, la porción que sea entregada debe estar completamente automatizada, de lo contrario se obtendrá una nota de 0.
 - En caso de no entregar documentación se obtendrá una nota de 0.
 - El email de entrega debe contener una copia del proyecto en formato tar.gz y un link al repositorio dónde se encuentra almacenado, debe seguir los lineamientos en el documento [Lineamientos para el envío de emails.pdf](#).
 - Los grupos de trabajo deben autogestionarse; la persona facilitadora del curso no es responsable si un miembro del equipo no realiza su trabajo.
 - En consultas presenciales, por correo electrónico o por Telegram, como mínimo se debe haber leído sobre el tema y haber intentado comprenderlo. Las consultas deben ser concretas y se debe demostrar que se intentó resolver el problema en cuestión.
 - El uso de Inteligencia Artificial (IA) estará regulado por los lineamientos establecidos en el documento [Política para el Uso de Inteligencia Artificial Generativa.pdf](#).
-

4. Descripción

Implementar un servidor utilizando el lenguaje de programación GNU C. El servidor deberá escuchar en el puerto TCP 9666. Cuando un cliente se conecte al servidor, podrá enviar las siguientes primitivas (mensajes):

Primitivas soportadas

1. Obtener dirección de broadcast

```
GET BROADCAST IP {dirección IP} MASK {máscara en formato /bits o notación X.X.X.X}
```

Ejemplos:

```
GET BROADCAST IP 10.8.2.5 MASK /29
```

Retorna: **10.8.2.7**

```
GET BROADCAST IP 172.16.0.56 MASK 255.255.255.128
```

Retorna: **172.16.0.127**

2. Obtener número de red

```
GET NETWORK NUMBER IP {dirección IP} MASK {máscara en formato /bits o notación X.X.X.X}
```

Ejemplos:

```
GET NETWORK NUMBER IP 10.8.2.5 MASK /29
```

Retorna: **10.8.2.0**

```
GET NETWORK NUMBER IP 172.16.0.56 MASK 255.255.255.128
```

Retorna: **172.16.0.0**

3. Obtener rango de hosts

```
GET HOSTS RANGE IP {dirección IP} MASK {máscara en formato /bits o notación X.X.X.X}
```

Ejemplos:

```
GET HOSTS RANGE IP 10.8.2.5 MASK /29
```

Retorna: **10.8.2.{1-6}**

```
GET HOSTS RANGE IP 172.16.0.56 MASK 255.255.255.128
```

Retorna: **172.16.0.{1-126}**

4. Obtener máscara o prefijo CIDR

```
GET NETMASK MASK {máscara en formato /bits o notación X.X.X.X}
```

Ejemplos:

```
GET NETMASK MASK /27
```

Retorna: **255.255.255.224**

```
GET NETMASK MASK 255.255.255.0
```

Retorna: **/24**

5. Obtener cantidad de hosts

```
GET HOSTS COUNT MASK {máscara en formato /bits o notación X.X.X.X}
```

Ejemplos:

```
GET HOSTS COUNT MASK /29
```

Retorna: **6**

```
GET HOSTS COUNT MASK 255.255.255.128
```

Retorna: **126**

6. Obtener clase de una dirección IP

```
GET IP CLASS {dirección IP}
```

Ejemplo:

```
GET IP CLASS 10.8.2.5
```

Retorna: **Clase A**

7. Generar subredes aleatorias

```
GET RANDOM SUBNETS NETWORK NUMBER {Y.Y.Y.Y} MASK {máscara en formato /bits o notación X.X.X.X} NUMBER {número de redes} SIZE {máscara en formato /bits o notación X.X.X.X}
```

Ejemplo:

```
GET RANDOM SUBNETS NETWORK NUMBER 10.0.0.0 MASK /8 NUMBER 3 SIZE /24
```

Puede retornar:

10.20.10.0/24 10.33.11.0/24 10.42.13.0/24

8. Agregar rutas

```
SET ROUTE {NAME} NETWORK NUMBER {Y.Y.Y.Y} MASK {máscara en formato /bits o notación X.X.X.X} PRIORITY {NUMBER}
```

9. Eliminar rutas

```
DEL ROUTE {NAME}
```

10. Realizar routing de una dirección IP

```
ROUTE IP {dirección IP} DEBUG {true/false}
```

Si no se especifica el parámetro **DEBUG**, su valor por defecto será false.

El modo debug deberá mostrar información adicional sobre el cálculo realizado.

Ejemplo:

```
ROUTE IP 8.8.8.8
```

Puede retornar:

nombre_de_ruta

Consideraciones

→ Únicamente se deberá implementar el servidor.

- Como cliente se utilizará el comando telnet.
 - Se deben validar errores y proporcionar mensajes de error útiles para el usuario.
-

5. Documentación

Se deberá entregar una documentación que, como mínimo, incluya los siguientes elementos:

- Instrucciones claras sobre cómo ejecutar la tarea corta.
- Pruebas realizadas, incluyendo los pasos necesarios para reproducirlas.
- Recomendaciones y conclusiones (al menos 10 de cada una).
- La documentación deberá cubrir todos los componentes implementados o instalados/configurados. En caso de que algún componente no haya sido implementado, no podrá ser documentado y esto tendrá un impacto en la completitud de la documentación.
- Referencias bibliográficas, cuando aplique.
- Una tabla donde se detalle qué funcionalidades funcionan y cuáles no funcionan dentro de la tarea corta.

Imaginen que la documentación está siendo creada para una persona con conocimientos básicos en el área de computación y que ustedes esperan que dicha persona pueda ejecutar la tarea corta y probarla sin ningún inconveniente. El uso de imágenes, diagramas, fragmentos de código (code snippets), videos y otros recursos visuales es altamente recomendado y puede resultar de gran utilidad.

La documentación deberá ser desarrollada en formato Markdown y posteriormente compilada en un archivo PDF.

6. Uso de GitHub y buenas prácticas de desarrollo de software

Cada grupo deberá aplicar los lineamientos especificados en el documento [Uso de GitHub.pdf](#). El incumplimiento de estos requerimientos tendrá un impacto en la nota final de la tarea corta.

A partir de la fecha de asignación de la tarea corta, cada grupo deberá entregar **todos los jueves** un reporte semanal de avance.

El reporte deberá incluir:

- La lista de tareas que se encuentran en desarrollo.
- La persona o personas asignadas a cada tarea.
- El estado y avance de cada tarea.
- Los problemas encontrados durante la semana.
- Una lista de los **commits** realizados por cada integrante, incluyendo su descripción y los archivos modificados.
- Una lista de los **Pull Requests (PRs)** creados por cada integrante, incluyendo su descripción y los cambios realizados.

Para el **primer reporte**, únicamente se deberá presentar la lista de tareas planificadas, una breve descripción de cada una y la persona o personas responsables de realizarlas.

A partir del **segundo reporte**, se deberá presentar el avance de cada tarea, los problemas encontrados y la evidencia del trabajo realizado mediante los commits y Pull Requests correspondientes.

El reporte deberá reflejar claramente la participación y el progreso individual de cada integrante del grupo.

Se deberá seguir el formato establecido en el documento [Reporte Avance.md](#).

En caso de que se extienda la fecha de entrega, se se deben entregar de avance adicionales.

El asunto (subject) del correo electrónico deberá utilizar el siguiente formato: **T2R{NÚMERO DE REPORTE}**. Por ejemplo, para el segundo reporte de avance, el asunto deberá ser **T2R2**.

7. Entregables

→ Documentación.

→ Helm Charts y todos los archivos/scripts requeridos para ejecutar la tarea.

8. Evaluación

Rubro	Valor
Documentación	10%
Implementación: Calculadora	30%
Reportes de avance y uso de GitHub	10%
Evaluación presencial	50%

9. Fechas importantes

→ **Entrega:** **06/10/26** antes de media noche.

→ **Evaluación presencial:** **13/10/26** durante la clase.

→ **Reportes de Avance:** Cada jueves, antes de la medianoche, a partir de la fecha de asignación de esta tarea corta.

10. Evaluación presencial

La evaluación presencial consiste en responder, de manera grupal, una serie de preguntas relacionadas con la implementación y la solución entregada. Su objetivo es demostrar el conocimiento, la comprensión y la autoría de la solución desarrollada por las personas estudiantes.

La evaluación tendrá una duración de **90 minutos**, es presencial y deberá realizarse con los mismos grupos de trabajo conformados para el desarrollo de la tarea.